Project Overlord

AI-Powered Cross-Chain Onboarding for Solana

---

1. Executive Summary

What We Are Building

Project Overlord is an AI-powered intent execution platform that removes the complexity of moving assets from Ethereum ecosystems (Base, Arbitrum, Ethereum, etc.) into Solana applications.

Instead of manually:

- bridging assets,
- swapping tokens,
- managing gas,
- understanding networks,
- and interacting with multiple UIs,

users simply express intent in natural language.

Example:

> "Put $100 from my Base wallet into SOL on Solana."

The AI agent autonomously: 1. understands the user's goal, 2. finds the optimal route, 3. bridges liquidity, 4. swaps assets if necessary, 5. delivers the correct asset on Solana, 6. optionally executes the destination action.

---

2. The Core Problem

Current Web3 Onboarding is Broken

A user with funds on:

- Base
- Arbitrum
- Ethereum
- Optimism

cannot easily access Solana apps.

The current onboarding flow is complicated: 1. Find a bridge 2. Understand supported chains 3. Transfer tokens 4. Wait for confirmations 5. Swap into the correct token 6. Handle gas requirements 7. Open a Solana wallet 8. Finally use the app

This creates:

- onboarding friction
- user dropoff
- failed conversions
- confusion for non-crypto-native users

---

3. Our Solution

Intent-Based Liquidity Onboarding

Instead of asking users:

> "How do you want to bridge?"

we ask:

> "What do you want to do?"

The AI handles the logistics.

Users interact with a conversational interface instead of complicated DeFi tooling.

---

4. Product Vision

AI-Powered Financial Intent Execution

Project Overlord transforms cross-chain onboarding into a simple conversational experience.

The user does not need to:

- understand bridges,
- know token standards,
- manage swaps manually,
- or understand chain infrastructure.

The system handles execution automatically after user confirmation.

---

5. Core User Experience

Chat-Based Interface

The application behaves like an AI assistant.

Example interaction:

```
User:
Put $50 from Base into SOL

AI:
I found the best route using LI.FI.
Estimated arrival: 45 seconds
Estimated fees: ~$1.20

[Confirm Transaction]
```

After confirmation:

```
Approving...
Bridging...
Swapping...
Completed ✓
```

This creates a much simpler onboarding experience compared to traditional DeFi flows.

---

6. Why a Chat Interface?

A conversational UI is strategically important because it:

- simplifies onboarding,
- strengthens the AI narrative,
- creates a futuristic product experience,
- improves demo quality,
- and reduces UI complexity.

Instead of building:

- dashboards,
- analytics terminals,
- or complicated DeFi panels,

we focus on:

- intent execution,
- transaction orchestration,
- and conversational UX.

---

7. Why Phantom Wallet?

We use Phantom because:

- it is the most recognized Solana wallet,
- it is developer-friendly,
- judges can test it easily,
- and it now supports multiple ecosystems.

Phantom currently supports:

- Solana
- Ethereum
- Base
- Polygon
- Bitcoin

This is important because users may only need ONE wallet for the full experience.

---

8. Main Features

8.1 AI Intent Parsing

The AI extracts:

- source chain,
- source token,
- amount,
- destination protocol,
- target chain,
- and intended action.

Example:

```
"Buy SOL using my Base USDC"
```

Extracted intent:

```
{
  "sourceChain": "Base",
  "sourceAsset": "USDC",
  "destinationChain": "Solana",
  "targetAsset": "SOL"
}
```

8.2 Cross-Chain Routing

The system uses LI.FI to:

- fetch bridge routes,
- aggregate swaps,
- optimize execution,
- and simplify liquidity movement.

8.3 Autonomous Transaction Orchestration

The AI:

- sequences transactions,
- prepares approvals,
- manages route execution,
- and updates the user during execution.

8.4 Solana App Onboarding

The platform can directly onboard users into:

- Drift,
- Jupiter,
- MarginFi,
- Tensor,
- Pump.fun,

- and other Solana applications.

8.5 Transaction Progress Tracking

Users can monitor:

- approvals,
- bridge status,
- swap completion,
- and final delivery.

---

9. Hackathon Track Alignment

Solana Main Track

We qualify because:

- Solana is central to the user journey,
- we deploy a Solana program,
- we improve onboarding into Solana applications,
- and we create a real utility product.

LI.FI Track

We strongly align with LI.FI because the project directly supports:

- cross-chain onboarding,
- AI-assisted transaction execution,
- and Solana liquidity routing.

---

10. System Architecture

High-Level Flow

```
User
 ↓
Chat Interface
 ↓
```

```
AI Intent Engine
 ↓
LI.FI Routing Layer
 ↓
Transaction Execution Layer
 ↓
Solana Destination
```

---

11. Technical Stack

Frontend

Framework

- Next.js

Styling

- TailwindCSS
- shadcn/ui

Wallet Integration

- Phantom Wallet
- Solana Wallet Adapter

Backend

Runtime

- Node.js

API Layer

- Next.js API routes OR

- Express.js

AI Layer

LLM Provider

Recommended:

- Claude OR
- OpenAI

AI SDK

- Vercel AI SDK

Optional:

- LangChain

Cross-Chain Infrastructure

Required

- LI.FI SDK OR
- LI.FI REST API

Optional:

- LI.FI MCP Server

Solana Layer

Smart Contract Framework

- Anchor

Language

- Rust

Solana SDK

- @solana/web3.js

Hosting

Frontend

- Vercel

Backend

- Railway OR

- Render

---

12. Recommended MVP Scope

IMPORTANT

Hackathons are won by:

- polished demos,
- smooth UX,
- and reliable execution.

Not by:

- giant infrastructure,
- huge protocol support,
- or overengineering.

MVP Goals

Support ONLY:

One Source Chain

Recommended:

- Base

One Destination Chain

- Solana

One Primary Flow

Examples:

- Base → SOL
- Base → Drift
- Base → Jupiter

This is enough for a strong demo.

---

13. What We NEED To Build

REQUIRED COMPONENTS

1. Frontend Chat UI

Users can:

- connect wallet,
- type intent,
- review route,
- and confirm execution.

---

2. AI Intent Parser

Convert user text into structured actions.

Example:

```
{
  "sourceChain": "Base",
  "amount": 50,
  "destinationChain": "Solana",
  "destinationAsset": "SOL"
}
```

---

3. LI.FI Integration

Use LI.FI to:

- fetch routes,
- prepare swaps,
- execute bridges,
- and optimize execution.

---

4. Solana Integration

Execute final delivery on Solana.

Examples:

- transfer SOL,
- deposit into Drift,
- complete swap.

---

5. Demo Flow

A complete working flow:

```
Base Wallet
 ↓
Bridge
 ↓
Swap
 ↓
Solana Destination
```

---

14. What We DO NOT Need To Build

DO NOT Build Your Own Bridge

Use LI.FI.

DO NOT Build Your Own Blockchain

Completely unnecessary.

DO NOT Support 20 Chains

Support:

- Base
- Solana

That is enough.

DO NOT Build Fully Autonomous AI Agents

We do not need:

- memory systems,
- autonomous wallets,
- self-learning AI,
- or autonomous trading.

The AI only needs to: 1. understand intent, 2. orchestrate execution, 3. explain actions, 4. and update status.

DO NOT Build a Token

Not needed.

DO NOT Build a DAO

Not useful for the hackathon.

DO NOT Build Complicated Backend Infrastructure

A lightweight architecture is enough.

---

15. Suggested Solana Program

Minimal Useful Program

Create an:

Intent Registry Program

The program stores:

- intent metadata,
- execution references,
- and transaction state.

This satisfies:

- Solana deployment requirements,
- and meaningful on-chain integration.

Without wasting development time.

---

16. AI Workflow

Step 1 — User Prompt

Example:

> “Move $100 from Base into SOL.”

Step 2 — Intent Extraction

AI returns:

```
{
  "fromChain": "Base",
  "toChain": "Solana",
```

```
  "token": "SOL",
  "amount": 100
}
```

Step 3 — Route Fetching

Backend calls:

- LI.FI SDK

Receives:

- route,
- gas estimate,
- transaction steps.

Step 4 — Execution

Wallet signs:

- approval transaction,
- bridge transaction.

Step 5 — Final Solana Action

Examples:

- swap into SOL,
- deposit into Drift,
- send to Solana wallet.

---

17. Suggested User Flows

Flow A — Basic Bridge

> “Move $50 to Solana.”

Flow B — Solana App Funding

> “Fund my Drift account.”

Flow C — Token Purchase

> “Buy BONK using my Base USDC.”

---

18. UI Pages

1. Landing Page

Explain:

- AI onboarding,
- one-click Solana access,
- and intent execution.

---

2. Main Dashboard

Primary chat interface.

---

3. Route Confirmation Screen

Display:

- bridge route,
- fees,
- estimated arrival time,
- and transaction summary.

---

4. Transaction Progress Screen

Example:

```
Approving...
Bridging...
```

```
Swapping...
Completed ✓
```

---

19. Demo Strategy

IMPORTANT

The demo should:

- feel magical,
- be easy to understand,
- and finish quickly.

Ideal Demo Flow

User Says:

> “Put $25 from Base into SOL.”

System:

- finds route,
- bridges assets,
- swaps tokens,
- and lands on Solana.

All within:

- 60–90 seconds.

---

20. GitHub Repository Structure

```
project-overlord/
│
├── frontend/
├── backend/
├── solana-program/
```

```
├── README.md
└── demo/
```

---

21. README Structure

The README should include:

- problem statement,
- solution overview,
- architecture,
- setup instructions,
- deployment instructions,
- demo instructions,
- screenshots,
- deployed Solana address.

---

22. Deliverables

REQUIRED

1. Public GitHub Repository

Open-source.

---

2. Solana Program Deployment

Deploy to:

- devnet.

---

3. Demo Video

Under:

- 3 minutes.

---

4. Live Demo

Deploy frontend to:

- Vercel.

---

5. Documentation

Detailed setup instructions and architecture.

---

23. Suggested Timeline

Day 1

- frontend setup,
- wallet integration,
- UI scaffolding.

Day 2

- AI intent parser,
- LI.FI integration,
- route fetching.

Day 3

- Solana integration,
- final transaction execution,
- transaction tracking.

Day 4

- polish UI,
- create demo video,
- deploy application,
- prepare submission.

24. Optional Stretch Features

ONLY if time remains:

- voice commands,
- mobile optimization,
- route comparison,
- gas sponsorship,
- AI transaction explanations,
- portfolio routing.

---

25. Final Positioning

Best Pitch Statement

> “Project Overlord is an AI-powered intent execution layer that eliminates cross-chain onboarding friction by automatically routing liquidity from any chain into Solana applications.”

---

26. Why This Project Can Win

The project combines:

- AI,
- conversational UX,
- cross-chain execution,
- Solana onboarding,
- and real user pain.

Most hackathon projects will build:

- dashboards,
- generic chatbots,
- or simple DeFi tooling.

Project Overlord focuses on:

> replacing complicated multi-step crypto workflows with intent-driven execution.

That creates a much stronger and more memorable product narrative.